Git 与 GitHub 速查

打印提示：A4、实际大小（100%）、关闭浏览器"页眉和页脚"；需要保留彩色底色时开启"背景图形"。用法：第 1 页背概念，第 2 页是每天要用的命令，第 3~4 页在卡住时翻。

本册关卡地图（先读这一页，1 分钟）

项目	内容
标签	**字典** —— 字典
本册关卡	—
用时	—（不占关卡：随查随用，不计时长）
学完解锁	每天 push 不断档

本册禁止通读，只许查；第 2 页天天要用（对应前置关 **0-02：Git 最小闭环**）。

本册是字典册、不占关卡：要查学习进度看《17-关卡总表》，本册只管"随查随用"。

开始之前 • 先弄懂这是什么（2 分钟）

① **是什么**：**Git** 是给代码"存档 + 时光机"的工具（在你本机跑）；**GitHub** 是把这些存档放到网上的仓库（别人能看、也能一起改）。生活类比：Git 像游戏里的**存档点**——每做完一小步存一次，改坏了能读档回去；GitHub 就是"把存档传到云端"，换台电脑也能接着玩。

② **干什么用**：写脚本、做项目时四件麻烦事全靠它 —— ① 改坏了要回退；② 想试新写法又怕搞砸（开分支）；③ 换电脑接着写（`git clone`）；④ 让别人和别人看到你每天在做什么：你的 GitHub 主页就是**最实在的一份学习履历**。

③ **为什么要学（+ 实战里怎么用）**：AI 运维交付的东西（脚本、compose 文件、训练代码）都用 Git 管理；GitHub 几乎是这行的硬通货；团队里天天要打交道的就是「代码怎么管理、多人怎么协作」。学习计划里"第 2 周末 commit ≥ 10 次"就靠这一册。

④ **长什么样 / 怎么用**：最小闭环一条循环：`git add → git commit → git push`（开工前先 `git pull`）。位置：字典类第 16 册——前面手写的每日日志、后面 10 册的综合项目，都往这个仓库里放。

读法：第 1 页先看三个概念和首次配置（一劳永逸）；配好之后每天只用第 2 页那六条命令。

目录

第 1 页	三个概念 + 首次配置（一劳永逸）
第 2 页	最小工作流：六条命令走完一个循环
第 3 页	撤销、查看、分支（出错时翻）
第 4 页	.gitignore + 常见报错 + 提交规范 + 小练习

第 1 页 • 三个概念与首次配置

一句话理解：Git = 给你的代码**拍快照**（随时能回到过去）；GitHub = 把快照**存到云端**（换电脑不丢、别人能看）。

看你 GitHub 仓库的人，看的就是提交记录——**那是你最真实的学习履历。**

1.1 三个概念（记住这个比喻就够）

概念	比喻	对应命令
仓库（repository）	一本带"存档功能"的相册	`git init`
提交（commit）	按下一次"存档"：记录这一刻的所有文件	`git add` + `git commit`

概念	比喻	对应命令
远程（remote）	云端备份（GitHub 上的那本相册）	`git push` / `git pull`

工作区 →（add）→ 暂存区 →（commit）→ 本地仓库 →（push）→ GitHub。四个格子，命令只在格子之间搬东西。

1.2 首次配置（只做一次，六条命令）

```
$ git --version                          # 没装就先：sudo apt install git
$ git config --global user.name "你的名字"
$ git config --global user.email "你的邮箱"      # 建议用 GitHub 注册邮箱
$ git config --global pull.rebase false      # 分叉时 pull 用合并（不加这条，Git 2.53+ 会报
"Need to specify how to reconcile divergent branches"）
$ ssh-keygen -t ed25519                  # 一路回车，生成 SSH 钥匙
$ cat ~/.ssh/id_ed25519.pub              # 复制这一整行 → 粘到 GitHub
```

粘贴位置：GitHub 网页 → 右上角头像 → **Settings** → **SSH and GPG keys** → New SSH key → 粘贴 → 保存。
验证：`ssh -T git@github.com` 出现 `Hi 你的用户名!` 就成了。

第 2 页 • 最小工作流：一个循环六条命令

日常循环（改代码 → 存档 → 上传），背下来：

```
$ git status                         # ① 看：哪些文件改了（最常用，随时敲）
$ git add .                          # ② 挑：把改动放进暂存区（. = 全部）
$ git commit -m "说明这次改了什么"      # ③ 存：拍快照
$ git push                           # ④ 传：同步到 GitHub
```

2.1 第一次：把本地目录变成仓库并推上去

```
$ cd ~/lab/myproject
$ git init                               # 变成 Git 仓库（多出一个 .git 隐藏目录）
$ git add . && git commit -m "首次提交"
$ git branch -M main                     # 主分支命名为 main（GitHub 的默认叫法）
$ git remote add origin git@github.com:你的用户名/仓库名.git
$ git push -u origin main                # 第一次要 -u，以后直接 git push
```

先在 GitHub 网页上点 **New repository** 建一个空仓库（**不要**勾选 "Add a README"，否则两边不一致会推不上去），再把上面的地址粘过来。

已经有仓库了（比如在网页建好、带着 README 的日志仓库）？ 不用 init / remote 那一串——一条命令把它拉到本地：**`git clone 仓库地址`**（地址在仓库页绿色 **Code** 按钮里）。拉下来之后，就回到上面的日常循环：add → commit → push。（0-02 关名里的「clone / add / commit / push」就是这条流）

推不动 / 拉不动？先看这里（网络兜底）：GitHub 在国内网络下时通时不通——
① 先在原命令上重试一次；还不行，换手机热点或换个时段。
② 走 SSH 方式（`ssh -T git@github.com` 验证）；若卡在连接超时（22 端口被挡）：GitHub 支持 SSH 走 443 端口——先 `ssh -T -p 443 git@ssh.github.com` 验证能通，再把 `Host github.com`、`Hostname ssh.github.com`、`Port 443` 三行写进 `~/.ssh/config`，之后照常用 `git@github.com:` 地址。
③ 实在连不上：换国内备选 Gitee——在 gitee.com 建好同名仓库，再把远端换掉：`git remote set-url origin 你的Gitee仓库地址`，然后 push 一次。
④ 第一次用 SSH 连接时会问 `Are you sure you want to continue connecting (yes/no)?`——输 `yes` 回车（首次校验服务器指纹，正常流程，不是报错）。

2.2 看一眼历史

```
$ git log --oneline                    # 一行一条提交记录（按 q 退出）
$ git diff                             # 看还没暂存的改动（改了哪几行）
```

第 3 页 • 撤销、查看、分支（出错时翻）

↩️ 改错了想撤回（按"改到哪一步"选）

情况	命令	说明
改了文件、还没 add	`git restore 文件名`	丢弃工作区改动，**不可找回**
已 add、还没 commit	`git restore --staged 文件名`	从暂存区拿出来，改动仍在
已 commit、想改说明	`git commit --amend -m "新说明"`	只改最近一次（还没 push 时用）
想回到某次提交	`git log --oneline`	先找到那次的编号（前 7 位），再决定怎么处理
已 push、想撤掉某次提交	`git revert 编号`	不改历史，新加一次"反做"提交把它抵消——**已推送时的正规退法**（编号用 git log 找）
想丢弃全部本地改动	`git reset --hard`	**危险**：所有未提交的改动全没

> ⚠️ 记住一条：**没提交的东西，Git 救不回来。**所以养成"小步提交"的习惯——改一点、存一次。

🌿 分支（先知道，别急着用）

```
$ git branch                           # 看有哪些分支（* 是当前）
$ git checkout -b feature-login          # 新建并切到新分支（试新想法不污染主线；新写法 git switch -c feature-login 同效）
$ git checkout main                    # 切回主线
$ git merge feature-login               # 把分支合并回主线
```

🔄 多人/多机同改：先拉后推

```
$ git pull                              # 开工前：把云端最新改动拉下来
$ git push                              # 收工前：把自己的改动推上去
```

口诀：**开工先 pull，收工再 push。**两台机器（或和同事）同时改，就不会打结。

第 4 页 • .gitignore • 常见报错 • 提交规范 • 小练习

🚫 .gitignore：哪些文件不提交

在仓库根目录建一个名为 `.gitignore` 的文件，写上不要提交的东西（每行一条）：

```
# 依赖与缓存（体积大、能重建）
.venv/
__pycache__/
*.pyc

# 日志与临时文件
*.log
tmp/

# 含密码的配置（千万别传！）
.env
.env.*
*.env
config_secret.*
```

> 📌 记住：**能自动重建的东西不提交**（.venv、__pycache__），**要重建它的清单要提交**（requirements.txt）。

🚗 六个最常见的报错（人话版）

报错	意思	怎么办
`fatal: not a git repository`	当前目录不是 Git 仓库	进到仓库目录；或先 `git init`
`nothing to commit, working tree clean`	没有新改动可提交	不是错误：你没改东西，或已经提交过了
`failed to push some refs`	云端有你本地没有的提交	先 `git pull`，再 `git push`；**若 pull 报** `Need to specify how to reconcile divergent branches`，先跑一次 `git config --global pull.rebase false`（首次配置里已含这条）
`Permission denied (publickey)`	GitHub 不认你的钥匙	重新把 `~/.ssh/id_ed25519.pub` 贴到 GitHub；或测试 `ssh -T git@github.com`
`CONFLICT (content): Merge conflict`	两边改了同一行	打开文件找 `<<<<<<<` 标记，人工留一份，然后 add + commit
`Repository not found`	GitHub 上找不到这个名字的仓库	仓库名/用户名敲错了，或**还没在 GitHub 上建这个空仓库**——照 2.1 建好再推

✏️ 提交信息怎么写（别人真的会看）

写法	示例	点评
❌ 太随意	`update` / `改了一下` / `aaa`	看不出做了什么
✅ 推荐	`新增磁盘巡检脚本` / `修复日志解析的空行报错`	一句话说清"做了什么"
✅ 进阶（可选）	`feat: 磁盘巡检脚本` / `fix: 日志解析空行`	加前缀：feat 新功能、fix 修复、docs 文档

🎯 小练习（做完你的 GitHub 就有内容了）

① 把 `~/lab` 下写好的脚本目录变成 Git 仓库；② 建好 `.gitignore`（按上面四类）；③ 提交 3 次以上（每次说明写清楚）；④ 先在 GitHub 网页建一个空仓库（照 2.1，别勾 README）；⑤ 推到 GitHub（两步命令见 2.1）；⑥ 在网页上打开仓库，确认能看到文件和历史记录。

> 📌 检查点对接：学习计划的"第 2 周末 commit ≥ 10 次""第 4 周末脚本进 GitHub"——就靠这一册完成。

每日日志怎么记（0-06 • 三行模板）

```
# 2026-10-13（周一）
- 做了什么：把 df / du 的示范各敲了一遍，日志里留 3 行复述
- 卡在哪：fuser 输出没看懂 → 查 man，看懂"占用进程"那列
- 明天第一件事：第 7 章挂载示范
```

就这三行，写进 `logs/2026-10/2026-10-13.md`（按月建目录），当天 commit 一次——连续 3 天，0-06 关就过了。

阶段总结练习（新增 • 打完本阶段关卡再做）

阶段 0 • 学习总结练习（关卡 0-07）：把日志仓库整理成"别人能看懂"的样子

业务场景：你换了台电脑，要在新机器上接着学——别人（或三个月后的你）打开你的 GitHub，必须一眼看出你每天在干什么。

综合要求：① 仓库首页写清"我在学什么、学到哪"；② 每天的日志按月归档（例如 `logs/2026-10/2026-10-13.md`）；③ 挑 3 天日志补上"卡在哪、怎么解决的"；④ 提交信息写得让人看得懂（不是 update）。

验收：□ 陌生人打开仓库能说出你在学什么　□ 连续 3 天有提交记录　□ 日志里至少 3 条"卡点 → 解决"

时间：60 分钟　**产出**：整理好的 learning-journey 仓库

AI 运维指向：运维日常就是"把过程写清楚"——故障记录、变更记录、交接文档都是这个能力。

实战价值：别人点开你的 GitHub，第一眼看到的就是这个。

附录 • 一个合格的 README 长什么样（4-10 要用）

这一页不占关卡、不占预算；写 README 之前回来照一遍就行。

README 是谁在看：看仓库的人、以及三个月后的你自己。他们的共同点是——**没耐心**，只想知道三件事：这是干什么的、怎么跑起来、跑出来长什么样。

段落	写什么
① **一句话说明**	这个仓库解决什么问题（例：三个 Shell 脚本，负责备份、磁盘告警、服务自愈）
② **怎么用**	按顺序可复制的命令：装什么、怎么跑、怎么验证（**别人照抄能跑通**是唯一标准）
③ **依赖与环境**	Ubuntu 26.04 / Python 3.14 / Docker 29.x（以你机器为准）；需要装什么
④ **效果**	一张截图或一段真实输出（证明你真的跑过）
⑤ **踩坑记录**	你遇到的两个坑和怎么解决的 —— 别人最爱看这一段

正例（照这个骨架写）

```
# shell-scripts

三个日常运维脚本：备份、磁盘告警、服务自愈。

## 怎么用
    git clone https://github.com/你的用户名/shell-scripts.git
    cd shell-scripts && chmod +x *.sh
    ./disk_alert.sh              # 磁盘超 80% 时输出告警
    THRESHOLD=90 ./disk_alert.sh # 自定义阈值

## 环境
Ubuntu 26.04、bash 5.3、Docker 29.x（都写"以你机器为准"也行），需要 df / tar / systemctl。

## 效果
    $ ./disk_alert.sh
    磁盘正常：根分区使用率 42%（示意，以你机器为准）

## 踩坑记录
- 定时任务里 ~ 不展开：改成绝对路径解决
- 文件名带空格时循环跑飞：用 IFS 和引号解决
```

三个最常见的毛病

① **只有标题没有用法**——别人不知道先敲哪条命令，等于没写；
② **没有依赖说明**——在你机器上能跑，在别人机器上报 command not found；
③ **没有效果证据**——只有源码没有输出/截图，别人会怀疑你是不是抄的。

写完自检一句话：**把一个陌生人放到这台机器上，照着 README 能不能跑通？**能，就算合格。
